

Advanced Day 6 — Async: Callbacks & Promises

Student Document · Advanced JS · Session 6 of 15

★ ADVANCED JS — DAY 6 OF 15

60 min

TOTAL

30 min

TEACHING

15 min

HANDS-ON

15 min

Q&A

Today's Topics

#	Topic
1	Sync vs Async
2	Callbacks: The Original Pattern
3	Callback Hell
4	Promise States and Creation
5	.then / .catch / .finally
6	Promise Combinators (all/race/allSettled/any)
7	Promise.resolve / Promise.reject

PART 1 — FOLLOW ALONG WITH YOUR INSTRUCTOR

Topic 1

Sync vs Async

JS doesn't wait

```
// SYNCHRONOUS – top to bottom, no surprises
console.log("1");
console.log("2");
console.log("3");
// Output: 1, 2, 3

// ASYNCHRONOUS – setTimeout schedules; doesn't pause
console.log("1");
```

```

setTimeout(() => console.log("2"), 0); // scheduled – runs LATER
console.log("3");
// Output: 1, 3, 2
//
// Even with delay 0! "2" goes through the event loop (Day 8).
// JS keeps running synchronous code first; async callbacks wait their turn.

```

Topic 2 Callbacks

Function passed to be called "when done"

```

// Simulate fetching a user from a server
function fetchUser(id, callback) {
  console.log(`Fetching user ${id}...`);
  setTimeout(() => {
    const user = { id, name: "Priya" };
    callback(null, user); // Node-style: (err, data)
  }, 1000);
}

fetchUser(7, (err, user) => {
  if (err) {
    console.error("Failed:", err);
    return;
  }
  console.log("Got user:", user);
});
// Logs:
//   Fetching user 7...
//   (1 second later)
//   Got user: { id: 7, name: "Priya" }

```

Topic 3 Callback Hell

The pyramid of doom

```

// The "pyramid of doom"
fetchUser(7, (err, user) => {
  if (err) { console.error(err); return; }

  fetchOrders(user.id, (err, orders) => {
    if (err) { console.error(err); return; }

    fetchItems(orders[0].id, (err, items) => {
      if (err) { console.error(err); return; }

      console.log(items); // finally got there!
    });
  });
});

```

```
});  
});  
  
// Problems:  
// 1. Indentation explodes – hard to read  
// 2. Error handling repeated at every level  
// 3. Hard to add steps without rewriting  
// 4. No way to return a value through this – only nest deeper
```

⚠ **This is why Promises exist.** The whole next decade of JS evolution was about flattening this pyramid.

Topic 4 Promise States

pending → fulfilled / rejected

State	Meaning
pending	Work in progress
fulfilled	Done, has a result
rejected	Failed, has an error
settled	Either fulfilled or rejected (terminal)

```
// Create a promise  
const p = new Promise((resolve, reject) => {  
  // do async work...  
  setTimeout(() => {  
    const success = true;  
    if (success) resolve("Done!"); // → fulfilled  
    else reject(new Error("Oops")); // → rejected  
  }, 1000);  
});  
  
// p is now PENDING. After 1s, it settles.  
  
// Real-world: wrapping an async API  
function delay(ms) {  
  return new Promise((resolve) => {  
    setTimeout(resolve, ms); // resolve with no value  
  });  
}  
  
delay(500).then(() => console.log("half a second passed"));
```

FOLLOW ALONG

```
const p = new Promise((res) => {
  setTimeout(
    () => res(42),
    100
  );
});
console.log(p);
p.then((v) => console.log(v));
```



first?
then?

Topic 5 .then / .catch / .finally

Chain it flat

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id < 0) reject(new Error("Bad id"));
      else resolve({ id, name: "Priya" });
    }, 500);
  });
}

// Chain – flat, not nested
fetchUser(7)
  .then((user) => {
    console.log("got user:", user);
    return user.id; // value passed to next .then
  })
  .then((id) => {
    return fetchUser(id + 1); // returning a Promise – chain awaits it
  })
  .then((nextUser) => {
    console.log("next user:", nextUser);
  })
  .catch((err) => {
    console.error("any failure caught here:", err.message);
  })
  .finally(() => {
    console.log("done – runs whether success or fail");
  });
```

Topic 6 Promise Combinators

Combine many into one

Combinator	Resolves when	Rejects when
<code>Promise.all</code>	ALL fulfill	ANY rejects (fail-fast)
<code>Promise.allSettled</code>	ALL settle	Never
<code>Promise.race</code>	FIRST settles	If first settles by rejecting
<code>Promise.any</code>	FIRST fulfills	ALL reject

```

const p1 = fetch("/user");
const p2 = fetch("/orders");
const p3 = fetch("/items");

// 1. all - wait for everything; fail if any fails
Promise.all([p1, p2, p3])
  .then(([user, orders, items]) => console.log("all done"))
  .catch((err) => console.error("at least one failed:", err));

// 2. allSettled - wait for everything, never throws
Promise.allSettled([p1, p2, p3])
  .then((results) => {
    results.forEach((r) => {
      if (r.status === "fulfilled") console.log("ok:", r.value);
      else console.log("fail:", r.reason);
    });
  });

// 3. race - first to settle wins (success OR fail)
Promise.race([p1, delay(5000)]) // timeout pattern
  .then((winner) => console.log("first done:", winner));

// 4. any - first to FULFILL wins; all-reject = AggregateError
Promise.any([p1, p2, p3])
  .then((firstSuccess) => console.log("got one:", firstSuccess))
  .catch((err) => console.error("everyone failed"));

```

Topic 7 Promise.resolve / .reject

Already-settled shortcuts

```

// Already-fulfilled promise
const cached = Promise.resolve({ id: 1, name: "Priya" });

cached.then((u) => console.log(u)); // logs the cached value

// Already-rejected promise
const failed = Promise.reject(new Error("nope"));
failed.catch((e) => console.log("caught:", e.message));

```

```
// Common pattern: cache or fetch
function getUser(id, cache) {
  if (cache[id]) return Promise.resolve(cache[id]); // instant
  return fetchUser(id); // real async
}

// Caller doesn't care which path — both return a Promise
getUser(7, {}).then(console.log);
```

PART 2 — HANDS-ON EXERCISE (15 MIN)

Task 1 Sync vs Async Output

- Type: `console.log("A"); setTimeout(() => console.log("B"), 0); console.log("C"); Promise.resolve().then(() => console.log("D"));`
- Predict the output order BEFORE running.
- Run. Note the actual order.
- In a comment, explain why D comes before B even though both are "async with delay 0".

Task 2 Promisify a Callback API

- Take this callback-style function: `function delayLog(msg, ms, cb) { setTimeout(() => { console.log(msg); cb(null); }, ms); }`
- Wrap it in a Promise-returning version `delayLogPromise(msg, ms)`.
- Use it to chain three logs: "1" after 300ms, then "2" after 200ms, then "3" after 100ms.

Task 3 Promise.all in Action

- Write a function `fetchPrice(item, ms)` that returns a Promise resolving to a price object after `ms` milliseconds. Use this data: `pen → 50, book → 200, bag → 800`.
- Use `Promise.all` to fetch all three in parallel.
- Sum the prices and log the total.
- Time the whole thing with `Date.now()` — confirm it's near the SLOWEST item, not the sum.

Bonus Promise.allSettled vs Promise.all

- Take three promises: `Promise.resolve("ok1")`, `Promise.reject(new Error("fail"))`, `Promise.resolve("ok2")`.
- First wrap them in `Promise.all` — what happens?

- Now wrap them in `Promise.allSettled` — what happens?
- In a comment, explain when you'd reach for each.

PART 3 — VISUAL SUMMARY

Promise Lifecycle

Stage	Can change?
pending	Yes — to fulfilled OR rejected
fulfilled	No — settled forever
rejected	No — settled forever

Handlers

Method	Runs on
<code>.then(fn)</code>	fulfilled
<code>.catch(fn)</code>	rejected
<code>.finally(fn)</code>	either

Combinators — Quick

Method	Best for
<code>.all</code>	Need ALL — abort on first fail
<code>.allSettled</code>	Need every outcome
<code>.race</code>	First-wins (timeouts)
<code>.any</code>	First success

Common Patterns

Pattern	Code
Delay	<code>new Promise(r => setTimeout(r, ms))</code>
Already-done	<code>Promise.resolve(value)</code>
Already-failed	<code>Promise.reject(error)</code>
Timeout	<code>Promise.race([job, timeout])</code>

HOMEWORK

Practice tasks before next class · Advanced Day 6

1. Write `wait(ms)` that returns a Promise resolving after `ms`. Use it: `wait(500).then(() => wait(500)).then(() => console.log("1s"))`.
2. Take a `fetchData(id)` that randomly succeeds or rejects (50/50). Chain three sequential `.then`s and end with `.catch`. Run multiple times to see both paths.
3. Use `Promise.race` with `[fetchUser(7), wait(2000).then(() => Promise.reject(new Error("timeout")))]` to add a 2-second timeout to a fetch.
4. Use `Promise.any` with three flaky fetches. Demonstrate that it returns the first SUCCESS, ignoring rejections.
5. Read: javascript.info/promise-basics and javascript.info/promise-chaining.
6. (Bonus) Watch Akshay Saini's "Namaste JS — Promises" episode. ~20 minutes.